

Universidad Nacional de Córdoba



Facultad de Ciencias Exactas, Físicas y Naturales

Cátedra de Arquitectura de Computadoras Trabajo Práctico 3: Pipeline RISC-V

Profesores: Martín Pereyra, Santiago Rodríguez

Integrantes:

Pallardó Agustín - DNI: 43.147.213 - apallardo@mi.unc.edu.ar

Trachta Agustín - DNI: 43.271.890 - agutrachta@mi.unc.edu.ar

24 de abril de 2026

Índice

1. Introducción	3
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
3. Arquitectura del Procesador RISC-V	5
3.1. Formato de las Instrucciones	5
3.1.1. Ejemplo de decodificación	6
3.2. Estructura del Pipeline	6
3.3. Unidad de Control y Decodificación	6
3.4. Unidad de Detección de Riesgos (Hazards) y Forwarding	7
3.4.1. Valores y Lógica de Forwarding	7
3.4.2. Detección y Resolución de Hazards (Stalls y Flushes)	7
3.5. Ejemplos Analíticos de Riesgos de Datos en Ensamblador	8
3.6. Módulo de Comunicación UART y FIFOs	9
4. Debug Unit	10
4.1. Rol de la Debug Unit dentro del sistema	10
4.2. Estados de la FSM	10
4.3. Protocolo de Comunicación	13
4.4. Modos de Operación	14
4.4.1. Modo Continuo (CMD_RUN)	14
4.4.2. Comando STOP	14
4.4.3. Modo Paso a Paso (CMD_STEP)	15
4.4.4. Señal HALT	15
4.5. Limpieza, Vaciado del Pipeline y Reinicio	15
4.6. Mecanismo de recepción UART y desacople de FIFO	16
4.7. Programación de memoria de instrucciones	16
4.8. Problemas encontrados durante el desarrollo	17
4.8.1. Confusión inicial entre pausa y terminación	17
4.8.2. Reanudación después de STOP	17
4.8.3. Problemas por programas demasiado cortos en el testbench	17
4.8.4. Cantidad de ciclos de drenado	18
4.9. Decisiones de diseño relevantes	18
4.9.1. Uso de ST_IDLE como estado de pausa	18
4.9.2. Separación entre pausa externa y fin interno	18
4.9.3. Reseteo controlado luego de cargar programa	19
4.9.4. Desacople entre recepción UART y decodificación de FSM	19
4.10. Validación mediante testbench	19
5. Resultados de Simulación	20
5.1. Casos de Prueba	20
5.2. Comportamiento ante la Ausencia de Instrucción HALT	20

6. Síntesis y Análisis de Temporización (Timing)	22
6.1. Reportes de Vivado	22
6.2. Identificación del Camino Crítico y Análisis Temporal	22
7. Conclusiones	26

1. Introducción

Un *pipeline* es una técnica de diseño de procesadores que busca mejorar el rendimiento dividiendo la ejecución de cada instrucción en una serie de etapas secuenciales. De este modo, varias instrucciones pueden avanzar simultáneamente por distintas fases del procesamiento, lo que permite un mejor aprovechamiento de los recursos del sistema.

La arquitectura *RISC-V*, basada en el paradigma *Reduced Instruction Set Computer*, se destaca por su simplicidad, modularidad y carácter abierto. Estas características la convierten en una opción especialmente adecuada para estudiar la organización interna de un procesador y desarrollar implementaciones escalables, verificables y de complejidad controlada.

En este trabajo se presenta el diseño, la implementación y la validación de un procesador *RISC-V* segmentado de cinco etapas sobre una FPGA Basys 3. El sistema desarrollado corresponde a un procesador de 32 bits organizado en las etapas *Instruction Fetch* (IF), *Instruction Decode* (ID), *Execute* (EX), *Memory Access* (MEM) y *Write Back* (WB).

El objetivo principal es construir un *pipeline* funcional capaz de ejecutar un subconjunto de instrucciones de la arquitectura *RISC-V*, garantizando el transporte correcto de datos y señales de control entre sus distintas etapas. Para ello, fue necesario diseñar tanto el camino de datos como la unidad de control, y resolver adecuadamente problemas asociados a dependencias de datos, riesgos de control y sincronización del flujo de ejecución.

Además, se incorporan mecanismos de observabilidad y depuración que permiten inspeccionar el estado interno del sistema durante su funcionamiento. Con este fin, se desarrolló una *Debug Unit* que se comunica con el procesador mediante comandos enviados por UART, posibilitando la carga de programas en memoria, la ejecución paso a paso, la detención controlada y el volcado de registros, memoria y registros inter-etapa del *pipeline*.

Este desarrollo integra módulos construidos en trabajos prácticos anteriores, en particular las unidades de ALU y UART (con leves modificaciones), incorporándolos dentro de una arquitectura más completa orientada no sólo a la ejecución de programas, sino también a su validación y depuración.

2. Objetivos

2.1. Objetivo general

Implementar y validar el pipeline de un procesador RISC-V en una FPGA Basys 3.

2.2. Objetivos específicos

- Implementar el pipeline del procesador RISC-V en Verilog, permitiendo la ejecución de instrucciones de manera correcta y eficiente.
- Desarrollar una Debug Unit que permita enviar y recibir información al procesador mediante el protocolo UART.
- Crear una interfaz para visualizar los datos e interactuar con la Debug Unit, utilizando CLI, GUI y/o TUI.
- Lograr que el procesador sea capaz de programarse y reprogramarse a través de comandos de UART.
- Garantizar que el clock no sea intervenido en ninguna parte del proyecto.
- Documentar decisiones tomadas en el diseño, justificando el razonamiento detrás de cada una.
- Creatividad al momento de mostrar los datos e interactuar con ellos (GUI, TUI, CLI).

3. Arquitectura del Procesador RISC-V

3.1. Formato de las Instrucciones

La arquitectura RV32I utiliza instrucciones de 32 bits de longitud fija. Según el tipo de instrucción, esos 32 bits se organizan en distintos campos, como `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` e inmediatos. Estos campos permiten identificar qué operación debe realizar el procesador, qué registros participan y si la instrucción requiere un valor inmediato.

Los formatos principales implementados en este procesador son: Tipo-R, Tipo-I, Tipo-S, Tipo-B, Tipo-U y Tipo-J.

Tipo	Campos
R	<code>funct7 rs2 rs1 funct3 rd opcode</code>
I	<code>imm[11:0] rs1 funct3 rd opcode</code>
S	<code>imm[11:5] rs2 rs1 funct3 imm[4:0] opcode</code>
B	<code>imm[12 10:5] rs2 rs1 funct3 imm[4:1 11] opcode</code>
U	<code>imm[31:12] rd opcode</code>
J	<code>imm[20 10:1 11 19:12] rd opcode</code>

Cuadro 1: Formatos principales de instrucciones en RV32I.

Cada formato responde a una necesidad distinta dentro del conjunto de instrucciones:

- **Tipo-R:** se utiliza para operaciones entre registros, como `add`, `sub`, `and`, `or`, `sll` o `slt`. En este caso, los campos `funct3` y `funct7` permiten distinguir la operación exacta.
- **Tipo-I:** se utiliza para instrucciones con inmediato, como `addi`, `andi`, `ori`, cargas desde memoria (`lb`, `lh`, `lw`, etc.) y también `jalr`. Incluye un inmediato de 12 bits.
- **Tipo-S:** se utiliza para escrituras en memoria, como `sb`, `sh` y `sw`. En este formato, el inmediato está dividido en dos partes dentro de la instrucción.
- **Tipo-B:** se utiliza para saltos condicionales, como `beq` y `bne`. El inmediato también aparece dividido, y luego debe reconstruirse para calcular la dirección de salto.
- **Tipo-U:** se utiliza en instrucciones como `lui`, donde se carga un inmediato de 20 bits en la parte superior del registro destino.
- **Tipo-J:** se utiliza en instrucciones de salto como `jal`, donde el inmediato permite calcular una dirección de destino más amplia.

Esta organización fija de los bits simplifica la etapa de decodificación, ya que permite extraer de manera ordenada los campos relevantes de cada instrucción y generar, a

partir de ellos, tanto las señales de control como los operandos necesarios para la ejecución.

3.1.1. Ejemplo de decodificación

Por ejemplo, la instrucción `add x5, x1, x2` corresponde al formato Tipo-R. En este caso:

- `opcode` indica que se trata de una operación aritmética entre registros,
- `rd = x5` señala el registro destino,
- `rs1 = x1` y `rs2 = x2` indican los registros fuente,
- `funct3` y `funct7` determinan que la operación exacta es una suma.

De forma similar, en instrucciones como `addi x5, x1, 10`, el formato Tipo-I conserva los campos `rd` y `rs1`, pero reemplaza `rs2` y `funct7` por un inmediato de 12 bits, que luego será extendido y utilizado por la ALU.

3.2. Estructura del Pipeline

El núcleo del procesador implementa una arquitectura segmentada de 5 etapas: IF, ID, EX, MEM y WB. Estas etapas están separadas por registros intermedios o *latches* (`if_id_reg`, `id_ex_reg`, `ex_mem_reg`, `mem_wb_reg`).

En el archivo `top.v` se realiza la interconexión de todas ellas. Una característica importante de esta arquitectura es que la resolución de saltos incondicionales (`JAL`, `JALR`) y condicionales (`BRANCH`) se realiza de forma temprana en la etapa de decodificación (ID). Esto puede verse en las señales de redirección generadas en ID (`redirect_target_id`), que alimentan directamente al multiplexor del PC en la etapa IF. Gracias a esto, se reduce la penalización cuando un salto es tomado, en comparación con otros diseños que resuelven estas instrucciones recién en la etapa EX.

3.3. Unidad de Control y Decodificación

El recorrido de los datos y el comportamiento del procesador dependen principalmente de la Unidad de Control. Su implementación se concentra en dos submódulos instanciados dentro de la etapa de decodificación (`id_stage.v`):

- `control_unit.v`: A partir del `opcode` de 7 bits, genera las señales de control de la ruta de datos. Por ejemplo, al detectar el opcode `7'b0000011`, correspondiente a una instrucción de carga (`LW`), activa `MemRead = 1`, indica que el dato que se escribirá en el banco de registros proviene de memoria (`MemToReg = 1`) y selecciona el inmediato como segundo operando de la ALU (`ALUSrc = 1`) para calcular la dirección efectiva.

- `rv_decoder.v`: Se encarga de la decodificación fina de las operaciones de la ALU. Interpreta los campos `funct3` y `funct7` de las instrucciones Tipo-R y Tipo-I para determinar qué operación exacta debe realizarse, como suma, resta, desplazamientos o comparaciones.

3.4. Unidad de Detección de Riesgos (Hazards) y Forwarding

La ejecución en pipeline introduce dependencias entre instrucciones que deben resolverse para mantener el funcionamiento correcto del procesador. Para ello, el diseño incluye dos mecanismos principales: *forwarding*, que adelanta resultados entre etapas, y *stalling*, que detiene temporalmente el avance del pipeline cuando no es posible resolver un conflicto de otra manera.

3.4.1. Valores y Lógica de Forwarding

El módulo `forwarding_unit.v` evita ciclos de espera innecesarios adelantando resultados desde etapas posteriores (EX/MEM o MEM/WB) hacia las entradas de la ALU en la etapa EX. Para esto, genera dos señales de selección: `fwdA` para el operando `rs1` y `fwdB` para el operando `rs2`.

En la siguiente tabla se resumen los valores utilizados:

Valor (<code>fwdA/B</code>)	Tipo de Forwarding	Acción del Mux
2'b00	Sin forwarding	Se usa directamente el valor proveniente del banco de registros almacenado en el latch ID/EX.
2'b10	Desde EX/MEM	Se toma el resultado adelantado desde EX/MEM. Esto ocurre cuando la instrucción inmediatamente anterior ya calculó el dato necesario.
2'b01	Desde MEM/WB	Se toma el valor adelantado desde MEM/WB. Se usa cuando no aplica forwarding desde EX/MEM.

Cuadro 2: Valores codificados para el MUX de forwarding de los operandos de la ALU.

3.4.2. Detección y Resolución de Hazards (Stalls y Flushes)

Cuando un conflicto no puede resolverse mediante forwarding, entra en acción el módulo `hazard_unit.v`.

En estos casos, la unidad detiene temporalmente el avance del pipeline activando `stall_if` y `stall_id`, lo que congela el Program Counter y la etapa de decodificación. Al mismo tiempo, puede inyectar una burbuja o *NOP* mediante `flush_idex = 1`. Las situaciones principales se resumen a continuación:

Clasificación	Dependencia	Solución aplicada
<i>Load-Use</i>	La instrucción en EX es un LW y su registro destino (rd) es utilizado inmediatamente por la instrucción en ID.	Se inserta 1 burbuja (+1 stall). Luego, el dato puede recuperarse mediante forwarding desde MEM/WB.
<i>Branch-EX</i>	Un Branch en ID necesita comparar un valor que todavía se está calculando en la etapa EX.	Se inserta 1 burbuja (+1 stall) para esperar a que el dato esté disponible.
<i>Branch-MEM</i>	Un Branch en ID depende de un dato cargado desde memoria y aún no disponible.	Se requieren 2 ciclos de espera .

Cuadro 3: Casos principales que generan stalls o flushes en el pipeline.

3.5. Ejemplos Analíticos de Riesgos de Datos en Ensamblador

Para mostrar de forma práctica cómo actúan estos mecanismos, se presentan algunos ejemplos en ensamblador RV32I.

Ejemplo 1: Forwarding continuo sin penalización

1. `add x1, x2, x3` ; Suma y guarda el resultado en x1
2. `add x4, x1, x5` ; Usa inmediatamente el valor de x1
3. `add x6, x1, x7` ; Vuelve a usar x1 en la instrucción siguiente

Sin forwarding, estas instrucciones generarían conflictos. Sin embargo, al ejecutar la línea 2, el valor de x1 todavía no fue escrito en el banco de registros, pero la unidad de forwarding detecta que está disponible en EX/MEM, por lo que activa `fwdA = 2'b10`. Luego, en la línea 3, ese mismo valor ya se encuentra más adelante en el pipeline, y se utiliza forwarding desde MEM/WB con `fwdA = 2'b01`. De esta forma, no es necesario detener la ejecución.

Ejemplo 2: Load-Use Hazard

1. `lw x1, 0(x2)` ; Carga un dato desde memoria en x1
2. `add x3, x1, x4` ; Usa inmediatamente el valor cargado en x1

En este caso, la instrucción `lw` recién obtiene el dato al final de la etapa MEM, por lo que la instrucción siguiente no puede usarlo de inmediato. Como no es posible resolver este caso con forwarding en el mismo ciclo, la unidad de hazards activa `stall_if = 1`, `stall_id = 1` y `flush_idx = 1`, insertando una burbuja en el pipeline. Luego de ese ciclo de espera, el valor ya puede ser adelantado mediante forwarding desde MEM/WB.

3.6. Módulo de Comunicación UART y FIFOs

La comunicación con el sistema externo se realiza mediante una interfaz UART asincrónica, compuesta por tres módulos principales: `baud_gen.v`, `uart_rx.v` y `uart_tx.v`.

Como el reloj interno del sistema trabaja a frecuencias mucho mayores que la velocidad de transmisión serie (base de 100 MHz), fue necesario incorporar buffers intermedios mediante `fifo.v`.

Estas FIFOs se implementan como memorias circulares con dos punteros: uno de lectura (`r_ptr_reg`) y otro de escritura (`w_ptr_reg`). Esto permite almacenar temporalmente los datos y desacoplar la velocidad de la CPU de la velocidad de la UART. Así, aunque el host envíe comandos en ráfaga, estos quedan correctamente almacenados y se procesan en orden, evitando pérdidas o sobrescrituras mientras la CPU continúa con otras tareas.

4. Debug Unit

Con el fin de poder observar, controlar y validar el funcionamiento del procesador implementado, se diseñó una unidad de depuración externa al pipeline. Su función principal es actuar como intermediaria entre el host y el procesador. Esta unidad funciona como la máquina de estados principal encargada de controlar la ejecución de la CPU. Su objetivo es permitir la carga de programas, la ejecución paso a paso, la detención del procesador y la lectura de registros, memoria y latches internos, todo a través de la interfaz UART.

La motivación principal para incorporar esta unidad fue crear un puente entre el usuario y la CPU, facilitando las pruebas y el análisis del sistema. En lugar de modificar directamente el núcleo para cada prueba, se definió una interfaz de control basada en comandos recibidos, de modo que un testbench o una aplicación externa puedan interactuar con el sistema de manera uniforme.

4.1. Rol de la Debug Unit dentro del sistema

La `debug_unit` se ubica por fuera del pipeline, pero conectada a señales clave de control. Entre sus responsabilidades se encuentran:

- Habilitar o detener la ejecución del pipeline mediante señales internas del pipeline.
- Generar un reinicio controlado de la ejecución.
- Limpiar y programar la memoria de instrucciones.
- Recibir comandos desde una FIFO de recepción UART sin romper el funcionamiento.
- Transmitir respuestas o estados por UART sin pisar la FIFO de transmisión.
- Permite la utilización de HALT como breakpoint y además como mecanismo de finalización de la ejecución.
- Gestiona mecanismos de lecturas de registros y memoria.

De esta forma, el procesador no necesita conocer detalles de la UART ni del protocolo de control; solamente expone ciertas entradas y salidas de depuración. Toda la secuencia de control de alto nivel queda encapsulada en una máquina de estados finitos dentro de la unidad de depuración.

4.2. Estados de la FSM

La FSM posee muchos estados fundamentales para su correcto funcionamiento, por lo que se especificarán a continuación separados por tablas:

Estado FSM	Bloque	Función
ST_IDLE	General	Estado de espera. Mantiene el pipeline detenido y aguarda la llegada de un nuevo comando.
ST_DECODE_CMD	General	Decodifica el comando recibido y selecciona la secuencia de estados correspondiente.
ST_ERROR	General	Maneja un comando inválido o una situación de error y envía una respuesta de error.
ST_RUN_CONTINUOUS	Ejecución	Habilita la ejecución continua del pipeline hasta recibir STOP o detectar HALT.
ST_STEP_ARM	Ejecución	Prepara un avance controlado de un paso habilitando transitoriamente el pipeline.
ST_STEP_EXEC	Ejecución	Completa el paso de ejecución, vuelve a frenar el pipeline y responde al host.
ST_RESET_EXEC	Ejecución	Aplica un reset de ejecución durante algunos ciclos para reinicializar el pipeline.
ST_DRAIN	Ejecución	Deja avanzar las instrucciones pendientes tras HALT hasta vaciar el pipeline.
ST_DUMP_DONE	Dumps	Envía la marca de finalización de un dump y retorna al estado ocioso.

Cuadro 4: Estados generales y de ejecución de la FSM de la `debug_unit`.

Estado FSM	Bloque	Función
ST_PROG_CLEAR	Programación	Recorre la memoria de instrucciones escribiendo NOPs para limpiarla.
ST_PROG_BEGIN	Programación	Inicializa la secuencia de carga del programa y prepara contadores y direcciones.
ST_PROG_LEN_HI	Programación	Lee el byte alto de la cantidad total de palabras del programa.
ST_PROG_LEN_LO	Programación	Lee el byte bajo de la cantidad total de palabras del programa.
ST_PROG_B0	Programación	Lee el primer byte de una instrucción de 32 bits.
ST_PROG_B1	Programación	Lee el segundo byte de una instrucción de 32 bits.
ST_PROG_B2	Programación	Lee el tercer byte de una instrucción de 32 bits.
ST_PROG_B3	Programación	Lee el cuarto byte de una instrucción de 32 bits.
ST_PROG_WRITE_WORD	Programación	Escribe en la memoria la palabra armada con los cuatro bytes recibidos.
ST_PROG_END	Programación	Finaliza la programación, envía confirmación y da paso al reset de ejecución.

Cuadro 5: Estados de programación de la memoria de instrucciones.

Estado FSM	Bloque	Función
ST_DUMP_REGS_HDR	Dump REGS	Envía la cabecera del volcado de registros e inicializa el índice de lectura.
ST_DUMP_REGS_SET_IDX	Dump REGS	Presenta el índice del registro que se va a leer.
ST_DUMP_REGS_LATCH	Dump REGS	Captura el valor del registro seleccionado en un registro interno.
ST_DUMP_REGS_SEND_B3	Dump REGS	Envía el byte más significativo del registro leído.
ST_DUMP_REGS_SEND_B2	Dump REGS	Envía el segundo byte del registro leído.
ST_DUMP_REGS_SEND_B1	Dump REGS	Envía el tercer byte del registro leído.
ST_DUMP_REGS_SEND_B0	Dump REGS	Envía el byte menos significativo del registro leído.
ST_DUMP_REGS_NEXT	Dump REGS	Avanza al siguiente registro o finaliza el dump si ya se enviaron todos.

Cuadro 6: Estados del volcado del banco de registros.

Estado FSM	Bloque	Función
ST_DUMP_MEM_RD_START_HI	Dump MEM	Lee el byte alto de la dirección inicial del dump de memoria.
ST_DUMP_MEM_RD_START_LO	Dump MEM	Lee el byte bajo de la dirección inicial del dump de memoria.
ST_DUMP_MEM_RD_COUNT	Dump MEM	Lee la cantidad de palabras de memoria que se deben enviar.
ST_DUMP_MEM_HDR	Dump MEM	Envía la cabecera del dump de memoria e inicializa índices y contadores.
ST_DUMP_MEM_SET_ADDR	Dump MEM	Coloca en la interfaz de memoria la dirección de la palabra a leer.
ST_DUMP_MEM_WAIT	Dump MEM	Espera un ciclo para permitir que la memoria entregue el dato.
ST_DUMP_MEM_WAIT_2	Dump MEM	Agrega un ciclo extra de espera por la salida sincrónica de la BRAM.
ST_DUMP_MEM_LATCH	Dump MEM	Captura la palabra leída de memoria en un registro interno.
ST_DUMP_MEM_SEND_B3	Dump MEM	Envía el byte más significativo de la palabra leída.
ST_DUMP_MEM_SEND_B2	Dump MEM	Envía el segundo byte de la palabra leída.
ST_DUMP_MEM_SEND_B1	Dump MEM	Envía el tercer byte de la palabra leída.
ST_DUMP_MEM_SEND_B0	Dump MEM	Envía el byte menos significativo de la palabra leída.
ST_DUMP_MEM_NEXT	Dump MEM	Avanza a la siguiente palabra o finaliza el dump si se alcanzó la cantidad pedida.

Cuadro 7: Estados del volcado de memoria de datos.

Estado FSM	Bloque	Función
ST_DUMP_LATCH_HDR	Dump LATCHES	Envía la cabecera del volcado de latches e inicializa el índice de lectura.
ST_DUMP_LATCH_SET_IDX	Dump LATCHES	Presenta el índice del latch del pipeline que se va a leer.
ST_DUMP_LATCH_LATCH	Dump LATCHES	Captura el contenido del latch seleccionado en un registro interno.
ST_DUMP_LATCH_SEND_B3	Dump LATCHES	Envía el byte más significativo del latch leído.
ST_DUMP_LATCH_SEND_B2	Dump LATCHES	Envía el segundo byte del latch leído.
ST_DUMP_LATCH_SEND_B1	Dump LATCHES	Envía el tercer byte del latch leído.
ST_DUMP_LATCH_SEND_B0	Dump LATCHES	Envía el byte menos significativo del latch leído.
ST_DUMP_LATCH_NEXT	Dump LATCHES	Avanza al siguiente latch o finaliza el dump cuando ya se enviaron todos.

Cuadro 8: Estados del volcado de registros inter-etapa del pipeline.

4.3. Protocolo de Comunicación

La comunicación se realiza mediante comandos y respuestas codificados en bytes hexadecimales. Los comandos son enviados desde el host hacia la FPGA, y las respuestas son enviadas desde la FPGA hacia el host como confirmación o devolución de datos.

Comando (Mnemónico)	Código Hex	Función
CMD_NONE	0x00	Comando nulo, sin efecto.
CMD_PROG_BEGIN	0x10	Indica el comienzo de la carga de un programa en memoria de instrucciones. Luego se envían los datos correspondientes.
CMD_PROG_END	0x11	Marca el final de la programación.
CMD_RUN	0x20	Inicia la ejecución continua del programa cargado.
CMD_STEP	0x21	Avanza la ejecución un paso de reloj.
CMD_STOP	0x22	Detiene la ejecución actual.
CMD_DUMP_REGS	0x30	Solicita el contenido completo del banco de registros.
CMD_DUMP_LATCHES	0x31	Solicita el estado interno de los registros del pipeline.
CMD_DUMP_MEM	0x32	Solicita el contenido de la memoria de datos.
CMD_CLEAR_IMEM	0x40	Limpia la memoria de instrucciones escribiendo instrucciones NOP.

Cuadro 9: Comandos de entrada del protocolo UART.

Respuesta (Acuse)	Código Hex	Significado
RESP_OK_CLEAR	0xC0	La memoria de instrucciones fue limpiada correctamente.
RESP_OK_PROG	0xC1	La carga del programa se realizó correctamente.
RESP_OK_STEP	0xC2	Se ejecutó correctamente un paso de reloj.
RESP_OK_STOP	0xC3	La CPU fue detenida correctamente.
RESP_OK_RUN_END	0xC4	El programa terminó su ejecución al detectar una instrucción HALT y vaciar el pipeline.
RESP_DUMP_*	0xD0-D2	Indican el inicio o desarrollo de una trama de volcado de datos.
RESP_ERR	0xEE	Indica que se recibió un comando inválido o inesperado.

Cuadro 10: Respuestas enviadas por la Debug Unit al host.

4.4. Modos de Operación

La Debug Unit controla distintos modos de funcionamiento de la CPU a través de su máquina de estados y de la señal `pipeline_en_r`.

4.4.1. Modo Continuo (CMD_RUN)

Cuando el host envía el comando 0x20, la Debug Unit pasa al estado `ST_RUN_CONTINUOUS` y habilita la ejecución continua del pipeline mediante `pipeline_en_r = 1`.

Si durante la ejecución se detecta una instrucción **HALT**, el sistema deja de incorporar nuevas instrucciones y pasa al estado `ST_DRAIN`. En este estado, el pipeline sigue avanzando únicamente para permitir que las instrucciones que ya estaban dentro terminen su recorrido de forma segura. Una vez que el pipeline queda vacío, se envía la respuesta `RESP_OK_RUN_END` al host.

4.4.2. Comando STOP

El comando `CMD_STOP` fue diseñado para pausar la ejecución, no para terminarla. La intención era que, al recibir un **STOP** por UART, el pipeline deje de avanzar y el procesador conserve su estado interno, incluyendo PC, registros, latches y cualquier otra información visible. Posteriormente, un nuevo `CMD_RUN` o `CMD_STEP` permite continuar desde ese mismo punto.

La implementación final consistió en que, al detectar `CMD_STOP` durante `ST_RUN_CONTINUOUS`, se consume el byte recibido, se desactiva `o_pipeline_en`, se transmite `RESP_OK_STOP` y se regresa a `ST_IDLE`. Como `ST_IDLE` mantiene el pipeline frenado, no fue necesario agregar un estado especial de pausa, ni tampoco se manipuló el reloj del sistema.

4.4.3. Modo Paso a Paso (CMD_STEP)

Cuando se envía el comando 0x21 (CMD_STEP), la Debug Unit habilita el pipeline solo durante un ciclo de reloj, permitiendo observar el avance exacto de una instrucción o de una transición interna de forma controlada.

La estrategia adoptada fue dividirlo en dos estados: ST_STEP_ARM y ST_STEP_EXEC. En el primero se habilita el pipeline; en el segundo se vuelve a deshabilitar y se responde con RESP_OK_STEP. La idea de esta separación fue asegurar un pulso limpio de habilitación, evitando que la acción de un único paso quedara ambigua por compartir demasiada lógica con RUN.

Este modo es especialmente útil para depuración, ya que permite analizar el comportamiento del procesador con mucho detalle, ciclo por ciclo.

4.4.4. Señal HALT

Además del control externo por UART, el procesador puede detectar internamente una condición de fin de programa, representada por la señal `i_halt`. Esta señal no proviene del host, sino de la lógica de decodificación o control del propio pipeline, y se activa cuando se reconoce una instrucción especial de parada.

Cuando `i_halt` se detecta durante ST_RUN_CONTINUOUS, no se detiene inmediatamente el pipeline. En cambio, se ingresa al estado ST_DRAIN. La razón es que, al momento de detectar HALT, todavía pueden existir instrucciones válidas en etapas posteriores del pipeline. Si se congelara la ejecución en ese mismo instante, algunas de esas instrucciones podrían no completar su *writeback*, generando resultados incorrectos o incompletos.

Por ese motivo se implementó una política de “drenado” del pipeline.

4.5. Limpieza, Vaciado del Pipeline y Reinicio

Antes de cargar o ejecutar un programa, es importante limpiar correctamente tanto la memoria de instrucciones como los registros internos del pipeline para evitar comportamientos erróneos.

- **Limpieza de Memoria de Instrucciones (ST_PROG_CLEAR):** Se activa mediante el comando 0x40. La Debug Unit recorre toda la memoria de instrucciones utilizando un índice interno (`clear_idx`) y escribe en cada posición la instrucción 32'h00000013, que corresponde a un NOP. Esto asegura que no queden instrucciones residuales de ejecuciones anteriores.
- **Reinicio del Pipeline (ST_RESET_EXEC):** Complementa el proceso de inicialización limpiando los latches y estados internos del pipeline mediante la señal `reset_exec_r = 1`. Su función es evitar que queden datos viejos en los registros intermedios y garantizar que cada nueva ejecución comience desde un estado limpio.

4.6. Mecanismo de recepción UART y desacople de FIFO

Uno de los primeros aspectos de diseño que requirió atención fue la interfaz con la FIFO de recepción. No resultaba conveniente consumir directamente `i_rx_data` en la misma lógica combinacional que decide el próximo estado, debido a que ello podía producir dependencias temporales delicadas entre el pulso de lectura y la validez efectiva del dato.

Para resolver esto, se implementó un pequeño desacople con tres señales y registros internos:

- **`rx_pop_pending`**: indica que en el ciclo anterior se solicitó una lectura a la FIFO.
- **`rx_byte_reg`**: almacena el byte efectivamente recibido.
- **`rx_byte_valid`**: indica que el byte almacenado es válido y todavía no fue consumido por la FSM.

La idea es la siguiente: cuando la FSM detecta que la FIFO no está vacía y no hay un byte pendiente, genera `o_rx_rd_en`. En el siguiente ciclo, el dato leído se captura en `rx_byte_reg` y se marca como válido. Luego, cualquier estado puede consumirlo de manera estable mediante `next_rx_consume`. Este pequeño buffer simplificó mucho la lógica de recepción y evitó errores de sincronización.

4.7. Programación de memoria de instrucciones

La carga del programa en la memoria de instrucciones se implementó como una secuencia explícita. Primero, el host puede enviar `CMD_CLEAR_IMEM` para llenar la memoria con instrucciones NOP. Esto resulta útil para garantizar que, aun fuera del rango del programa cargado, el procesador no ejecute contenido residual.

A continuación, `CMD_PROG_BEGIN` inicia la carga propiamente dicha. El protocolo adoptado transmite:

1. Dos bytes con la cantidad total de palabras a grabar;
2. Luego, por cada instrucción, cuatro bytes correspondientes a la palabra de 32 bits.

La FSM recorre los estados `ST_PROG_LEN_HI` y `ST_PROG_LEN_LO` para reconstruir la longitud, y luego los estados `ST_PROG_B0` a `ST_PROG_B3` para reconstruir cada instrucción en `word_buffer`. Finalmente, en `ST_PROG_WRITE_WORD` se genera un pulso de escritura sobre la memoria de instrucciones y se actualiza la dirección de programación.

Una vez cargado el programa, se ingresa en `ST_PROG_END`, desde donde se responde al host y luego se fuerza una secuencia de reset de ejecución controlada mediante `ST_RESET_EXEC`.

4.8. Problemas encontrados durante el desarrollo

4.8.1. Confusión inicial entre pausa y terminación

Uno de los primeros problemas conceptuales surgió al definir qué debía significar exactamente `CMD_STOP`. Inicialmente aparecía la duda de si al detener el pipeline se debía también dar por terminada la ejecución. Sin embargo, eso entraba en conflicto con la idea de poder reanudar luego con `CMD_RUN` o continuar paso a paso con `CMD_STEP`.

La solución adoptada fue separar claramente ambos conceptos:

- `STOP` implica pausa externa y preservación completa del estado.
- `HALT` implica fin interno, seguido de drenado del pipeline.

Esta separación permitió estabilizar la diferencia de los comandos y facilitó el diseño para el testbench.

4.8.2. Reanudación después de `STOP`

Otro problema importante apareció cuando se intentó ejecutar la secuencia `RUN + STOP + RUN`. El objetivo era que el primer `RUN` hiciera avanzar el procesador, que `STOP` lo congelara, y que un segundo `RUN` permitiera continuar sin perder el estado previo.

Los fallos observados mostraban que, en algunos casos, luego del `STOP` el sistema parecía quedar en un estado desde el cual no lograba completarse una nueva ejecución, o bien el testbench esperaba una finalización que nunca llegaba. El análisis de los trazados permitió ver que una causa frecuente no era necesariamente un error de la FSM de pausa, sino el hecho de que el programa de prueba ya había alcanzado `HALT` antes de que llegara el `STOP`. En ese escenario, el `STOP` no estaba interrumpiendo una ejecución activa, sino actuando cuando el procesador ya había terminado naturalmente.

La solución fue rediseñar el caso de prueba. En lugar de usar programas muy cortos que alcanzaban `HALT` casi inmediatamente, se definió un programa con bucle infinito que incrementa un registro en cada iteración. De esta manera, se pudo verificar realmente el comportamiento de pausa y reanudación: tras el primer `STOP`, el valor del registro quedaba congelado; tras el segundo `RUN`, el valor volvía a incrementarse. Esta evidencia confirmó que la semántica de pausa estaba correctamente implementada.

4.8.3. Problemas por programas demasiado cortos en el testbench

Relacionada con la dificultad anterior, se detectó que varios fallos reportados por el testbench no provenían del diseño de la `debug_unit`, sino de la propia metodología de prueba. Programas de muy pocas instrucciones podían finalizar antes de que se enviara el comando `STOP`, lo cual generaba una interpretación engañosa del error.

Para evitar esto, fue necesario revisar la estrategia de validación. Se decidió que:

- los tests de `HALT` deben usar programas finitos y verificar el ingreso a `ST_DRAIN`,

- los tests de **STOP** deben usar programas suficientemente largos o infinitos, de modo que el comando efectivamente interrumpa una ejecución en curso,
- las rutinas del testbench que esperan **ST_IDLE** deben contemplar que el sistema puede haber pasado antes por estados transitorios como **ST_DRAIN**.

Esta separación entre casos de prueba mejoró significativamente la capacidad de diagnóstico.

4.8.4. Cantidad de ciclos de drenado

Otro punto que exigió ajuste fue la cantidad de ciclos requeridos en **ST_DRAIN**. Si el número era insuficiente, algunas instrucciones no alcanzaban a completar **WB**; si era excesivo, el comportamiento seguía siendo correcto pero menos preciso y más difícil de justificar formalmente.

La solución práctica inicial fue ajustar el contador de drenado observando trazas del pipeline y verificando en testbench que todos los resultados esperados quedaran escritos antes de declarar el fin del programa. Sin embargo, posteriormente se adoptó un criterio más robusto y preciso, reemplazando ese esquema fijo por una condición estructural basada en la detección explícita de vaciado del pipeline. En la implementación actual, la finalización del drenado se determina cuando todos los registros inter-etapa han finalizado, lo que permite concluir la ejecución de manera más fundamentada y consistente con el estado real del procesador.

4.9. Decisiones de diseño relevantes

4.9.1. Uso de **ST_IDLE** como estado de pausa

Como se mencionó, se decidió no crear un estado adicional de *paused*. En su lugar, **ST_IDLE** cumple dos funciones: esperar nuevos comandos y mantener el pipeline detenido. Esta decisión simplificó el diseño y fue consistente con el comportamiento deseado.

4.9.2. Separación entre pausa externa y fin interno

Distinguir **STOP** de **HALT** fue una de las decisiones más importantes del diseño. Sin esta separación, el comportamiento de reanudación habría sido ambiguo y el testbench mucho más difícil de estructurar. A nivel conceptual, **STOP** es una intervención del depurador que frena completamente la ejecución donde quedó; **HALT** es una propiedad del programa que se está ejecutando, que se utiliza tanto para el fin del programa como para breakpoints internos.

4.9.3. Reseteo controlado luego de cargar programa

Luego de programar la memoria de instrucciones, se resolvió forzar un reset de ejecución antes de permitir cualquier RUN. Esto garantizó que el procesador arranque desde un estado limpio y consistente con el programa recién cargado. Si no se hiciera esto, podrían persistir restos de una ejecución anterior en los registros inter-etapa.

4.9.4. Desacople entre recepción UART y decodificación de FSM

El uso del buffer `rx_byte_reg` con `rx_byte_valid` evitó depender de tiempos implícitos entre la FIFO y la lógica combinacional. Esta decisión mejoró la robustez y facilitó la depuración.

4.10. Validación mediante testbench

La validación se realizó mediante testbenches orientados a escenarios de uso reales. Entre los más importantes se incluyeron:

- limpieza de memoria de instrucciones,
- carga de programa y verificación de contenido,
- ejecución continua hasta `HALT`,
- secuencia `RUN + STOP + RUN`,
- programas con bucle infinito para validar pausa y reanudación efectiva.

Uno de los resultados más representativos fue el ensayo con un lazo infinito que incrementa un registro. Durante la ejecución continua, el valor del registro aumentaba de forma sostenida. Tras enviar `STOP`, dicho valor quedaba congelado. Luego, al enviar un nuevo `RUN`, el conteo continuaba desde el valor previamente alcanzado. Esta prueba permitió concluir que la pausa preserva correctamente el estado del procesador, que la reanudación funciona como se esperaba y que el reloj no se interviene en ningún momento.

5. Resultados de Simulación

5.1. Casos de Prueba

Para validar el funcionamiento del procesador se desarrollaron distintos casos de prueba en simulación, tanto a nivel de módulos individuales como del sistema completo. En una primera etapa, se verificó el comportamiento de bloques fundamentales como la ALU, la memoria de datos, la memoria de instrucciones, la unidad de forwarding y la unidad de detección de hazards. Luego, se realizaron pruebas integradas sobre el pipeline completo utilizando programas en ensamblador RV32I especialmente diseñados para ejercitar los distintos caminos de ejecución.

Los casos considerados incluyeron operaciones aritméticas y lógicas entre registros, instrucciones con inmediato, accesos a memoria mediante `load` y `store`, y secuencias con dependencias de datos entre instrucciones consecutivas. También se probaron escenarios con riesgos del tipo *load-use*, dependencias en instrucciones de salto condicional y situaciones donde el forwarding resultaba suficiente para evitar detenciones innecesarias del pipeline.

Además, se verificó el funcionamiento de la *Debug Unit* mediante testbenchs que simulaban el intercambio de comandos por UART. Esto permitió comprobar la correcta recepción de órdenes como carga de programa, ejecución continua, ejecución paso a paso, detención, limpieza de memoria y volcado del estado interno del procesador. Estas pruebas fueron especialmente útiles para detectar errores de integración entre módulos que no aparecían al analizar cada bloque por separado.

Un aspecto importante del proceso de simulación fue la necesidad de depurar problemas reales de temporización lógica y de sincronización funcional. Entre ellos se destacaron la correcta propagación de datos a través del pipeline, la validación del vaciado completo ante una instrucción `HALT`, y la correspondencia entre la latencia esperada de la memoria y el momento en que los datos debían estar disponibles para escritura o forwarding.

5.2. Comportamiento ante la Ausencia de Instrucción `HALT`

Las simulaciones permitieron comprobar que el procesador mantiene un comportamiento estable aun cuando el programa cargado no contiene una instrucción `HALT`. En este caso, la *Debug Unit* nunca detecta la condición de finalización del programa, por lo que la CPU continúa ejecutando instrucciones de manera indefinida mientras permanezca habilitada en modo continuo.

Sin embargo, este comportamiento no produce fallos internos ni estados inválidos. Esto se debe a que, antes de programar una nueva secuencia de instrucciones, la memoria de instrucciones es limpiada mediante el estado `ST_PROG_CLEAR`, reescribiendo todas sus posiciones con instrucciones `NOP`. Como resultado, si el contador de programa avanza más allá del código útil cargado por el usuario, el procesador continúa leyendo instrucciones neutras y el pipeline sigue operando de forma predecible.

En estas condiciones, el *Program Counter* continúa incrementándose hasta recorrer toda la memoria direccionable y eventualmente reiniciar su recorrido por desborde natural, sin provocar excepciones ni comportamientos erráticos dentro de la arquitectura implementada. Desde el punto de vista funcional, esto significa que la ejecución no termina por sí sola, pero el sistema sigue siendo controlable desde el exterior.

Este punto fue relevante durante la etapa de pruebas, ya que permitió confirmar que la finalización del programa no debía depender únicamente del agotamiento del código cargado, sino de una condición explícita de parada. También ayudó a validar la necesidad de contar con mecanismos externos de control, como `CMD_STOP`, para abortar la ejecución en cualquier momento desde la interfaz de depuración.

A su vez, el análisis de este caso permitió ajustar el comportamiento del sistema ante la detección de `HALT`. En particular, fue importante distinguir entre detener inmediatamente el ingreso de nuevas instrucciones y permitir que las que ya estaban dentro del pipeline terminaran su recorrido. Esta dificultad llevó a refinar la lógica de vaciado dinámico del pipeline, evitando soluciones rígidas basadas únicamente en una cantidad fija de ciclos.

6. Síntesis y Análisis de Temporización (Timing)

6.1. Reportes de Vivado

Una vez validado el diseño a nivel funcional mediante simulación, se procedió a su síntesis e implementación en Vivado con el objetivo de evaluar el costo en recursos y su comportamiento temporal sobre FPGA. Los reportes generados por la herramienta permitieron analizar la utilización de LUTs, registros, memoria embebida y rutas críticas, así como también verificar si el sistema cumplía con las restricciones de reloj planteadas.

Este análisis no solo permitió comprobar si el diseño era sintetizable, sino también detectar puntos sensibles en la implementación. En particular, fue importante estudiar cómo impactaban sobre el timing la interconexión entre etapas del pipeline, la lógica de forwarding, la unidad de hazards y el acceso a memorias. También se revisaron advertencias metodológicas y de temporización que surgieron durante el flujo de implementación, ya que varias de ellas ayudaron a identificar zonas del diseño que requerían una revisión más cuidadosa.

Otro aspecto que debió considerarse fue la correspondencia entre la memoria inferida o instanciada y la latencia asumida por el pipeline. Durante el desarrollo, una de las dificultades más importantes estuvo asociada justamente a la lectura de memoria de datos, ya que una discrepancia entre la latencia esperada y la configurada en el bloque de memoria podía desplazar los datos un ciclo completo y producir resultados incorrectos en la etapa de escritura.

6.2. Identificación del Camino Crítico y Análisis Temporal

Para la validación temporal del diseño se utilizó un reloj interno generado mediante el bloque **Clocking Wizard**, a partir del reloj base de la placa. Este mismo se tuvo que utilizar dado que en las primeras implementaciones no se pudo utilizar el reloj base porque no cumplía con los requisitos de timing. Durante las distintas etapas del desarrollo se trabajó con varias configuraciones de frecuencia, observándose que, a medida que se incorporaban nuevas funcionalidades al procesador —como lógica adicional de control, señales de depuración, mecanismos de volcado por UART y mayor interconexión entre módulos—, el cierre temporal se volvía más exigente. En consecuencia, fue necesario reducir progresivamente la frecuencia de operación hasta alcanzar una configuración estable de **72 MHz**, adoptada finalmente para el funcionamiento del sistema.

Este comportamiento es esperable en un diseño de este tipo, ya que el agregado de nuevas señales y caminos de control no sólo incrementa la profundidad lógica de ciertos recorridos, sino también el costo de ruteo y el *fanout* de señales críticas. Por ese motivo, la frecuencia máxima práctica no quedó determinada únicamente por la lógica funcional del pipeline, sino también por la complejidad de integración alcanzada en la implementación final.

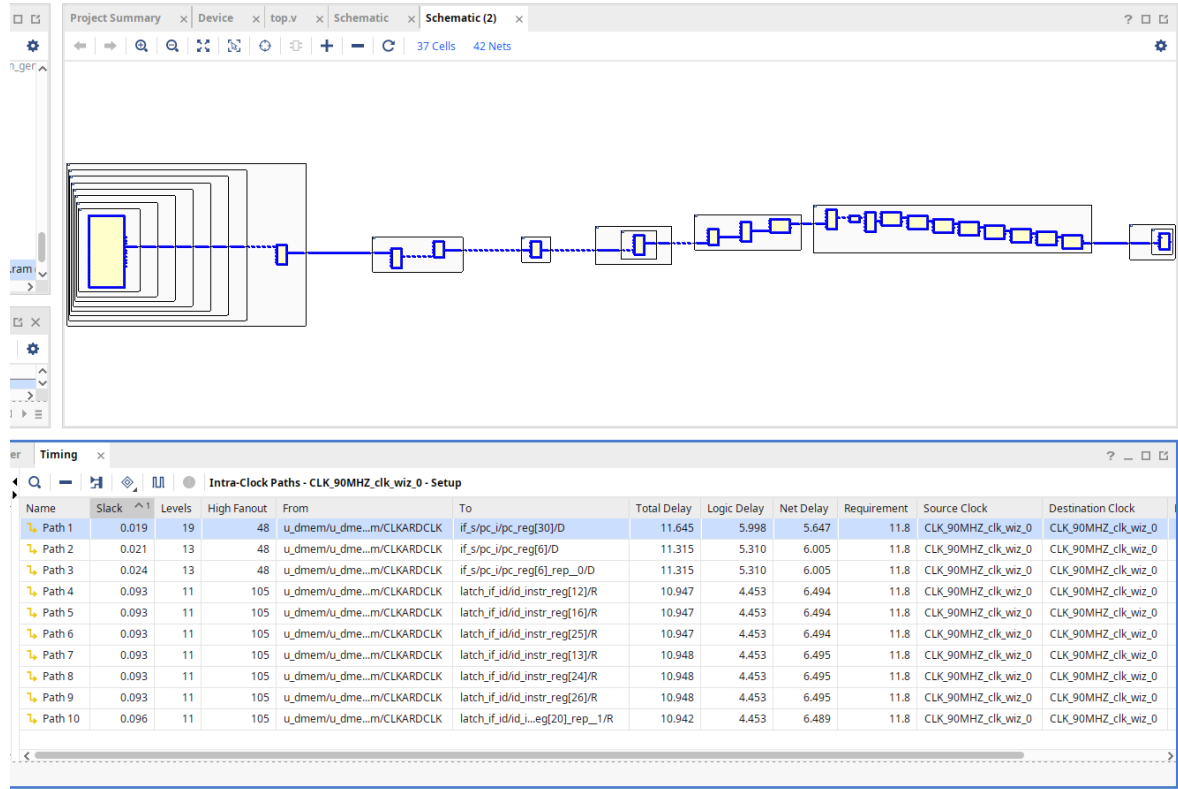


Figura 1: Camino crítico identificado por Vivado, comprometiendo principalmente a las etapas MEM, WB, ID e IF del pipeline.

El análisis del camino crítico mostró que el retardo más exigente del sistema no se concentra en un único bloque aislado, sino en un recorrido que enlaza varias partes relevantes de la arquitectura. De acuerdo con el esquema del camino crítico reportado por Vivado, dicho recorrido parte desde la memoria de datos, atraviesa la lógica de selección del dato de escritura de la etapa de *write back*, interactúa con el banco de registros y la etapa de decodificación (*regfile32*, *id_stage*) y finalmente alcanza la lógica de actualización del contador de programa y registros vinculados a la etapa de *instruction fetch* (*pc* e *if_id_reg*).

Desde el punto de vista arquitectónico, esto implica que en el camino crítico intervienen principalmente las etapas **MEM**, **WB**, **ID** e **IF**. En otras palabras, el dato leído desde memoria debe propagarse hasta la lógica de escritura de registros, afectar señales utilizadas durante la decodificación y participar luego en la lógica que condiciona la actualización del PC y de los registros inter-etapa. Esta extensión del recorrido explica por qué el retardo total no depende únicamente de operadores aritméticos o multiplexores, sino también de la propagación a través de varios módulos conectados entre sí.

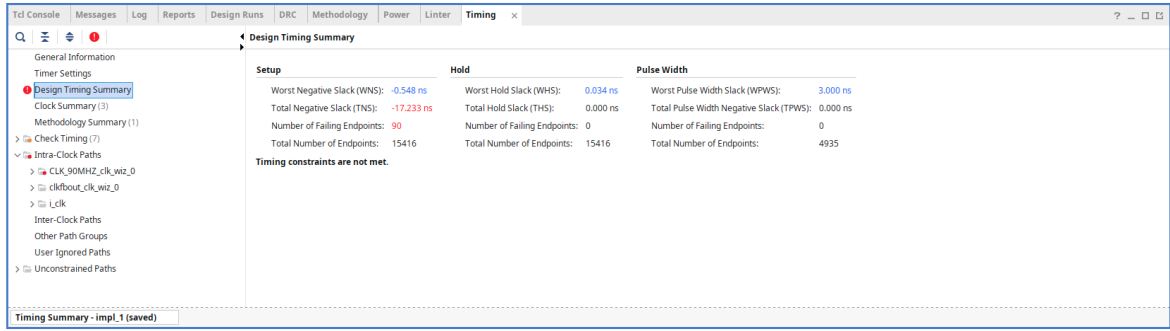


Figura 2: Resumen temporal de implementación en una configuración que no alcanzaba a cumplir las restricciones impuestas. Se observa una violación de *setup*, con *Worst Negative Slack* negativo y múltiples endpoints en falla.

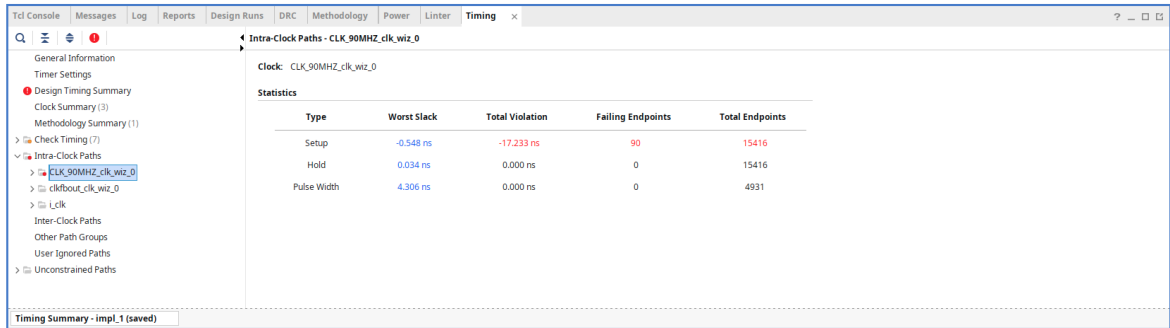


Figura 3: Resumen de caminos *intra-clock*. Se observa que las violaciones temporales se concentraban dentro de un mismo dominio de reloj.

Los reportes mostraron que el retardo total del camino crítico se reparte de manera comparable entre *logic delay* y *net delay*. Esto indica que el problema no se debe sólo a la profundidad combinacional, sino también al costo de interconexión física entre bloques, aspecto particularmente importante cuando el diseño crece en complejidad y el ruteo comienza a tener un peso similar al de la propia lógica.

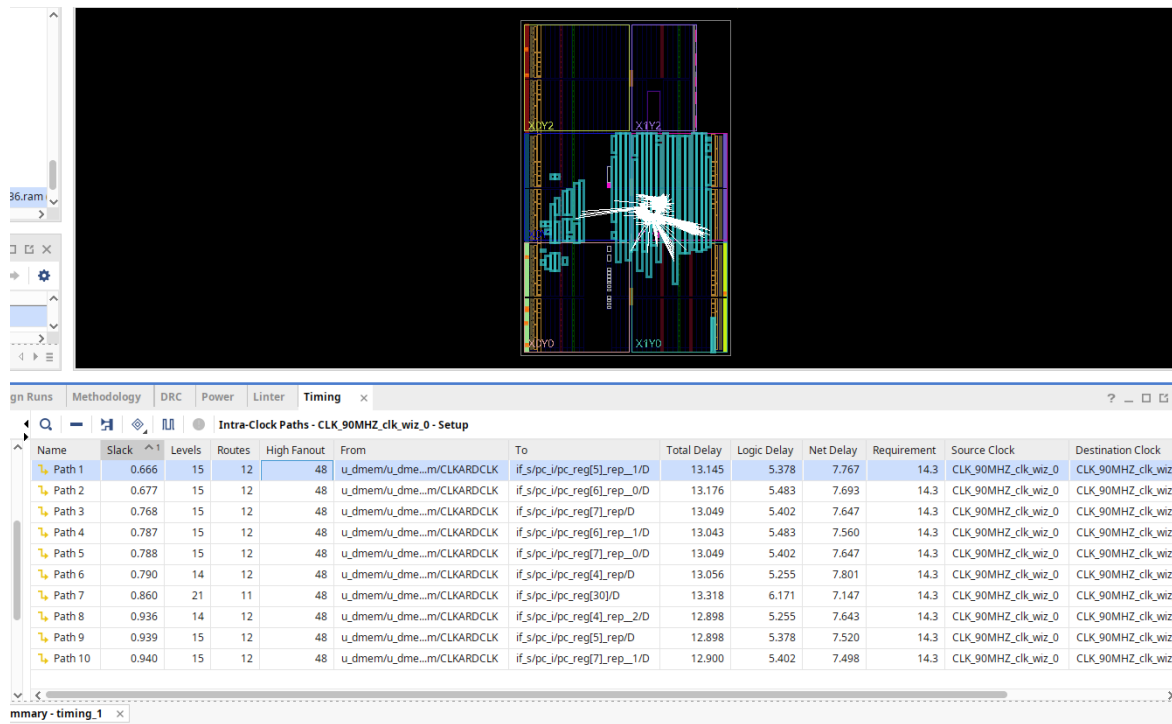


Figura 4: Visualización del ruteo.

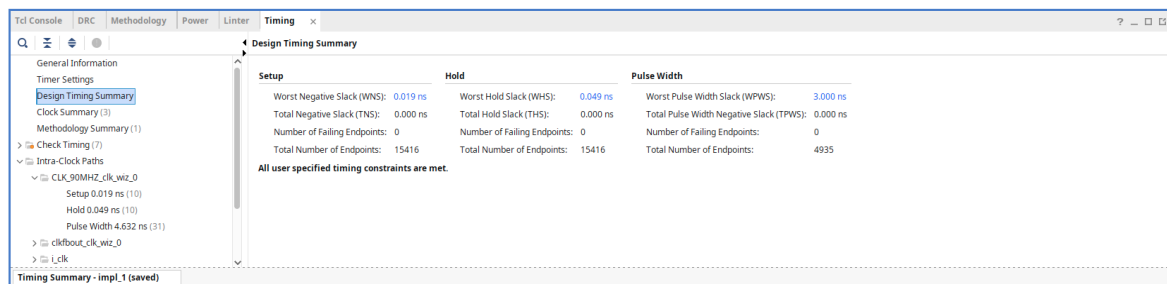


Figura 5: Resumen temporal de la implementación final. Se verifica que todas las restricciones temporales especificadas se cumplen, con *Worst Negative Slack* positivo y sin *endpoints* en falla.

En este contexto, la selección final de **72 MHz** representó un compromiso razonable entre rendimiento y robustez temporal. Esta configuración permitió conservar el comportamiento correcto del pipeline, la interacción con memorias y el funcionamiento de la unidad de depuración, manteniendo un margen temporal más adecuado para la implementación completa del sistema.

7. Conclusiones

El desarrollo de este trabajo permitió trasladar a una implementación concreta sobre FPGA varios de los conceptos centrales de arquitectura de computadoras estudiados durante la cursada. La construcción de un procesador *RISC-V* segmentado en cinco etapas implicó no sólo diseñar el camino de datos y la lógica de control, sino también resolver problemas reales de integración entre módulos, temporización, manejo de riesgos y validación funcional, que exceden el análisis puramente teórico de la arquitectura.

Uno de los aspectos más relevantes del proyecto fue la incorporación de una *Debug Unit* como mecanismo de control y observabilidad del sistema. Esta unidad permitió desacoplar la lógica de ejecución del procesador de las tareas de depuración, posibilitando la carga de programas por UART, la ejecución continua o paso a paso, la detención controlada y el volcado del estado interno del sistema. Contar con esta herramienta resultó fundamental para validar el comportamiento del procesador, detectar errores de manera sistemática y facilitar el análisis del tránsito de instrucciones a través del *pipeline*.

Desde el punto de vista funcional, la implementación de mecanismos de *forwarding*, *stall* y *flush* permitió resolver adecuadamente los principales riesgos asociados a una arquitectura segmentada. Las pruebas realizadas mostraron que el procesador mantiene la correctitud de ejecución frente a dependencias de datos, conflictos del tipo *load-use* y cambios de flujo producidos por saltos y bifurcaciones. Del mismo modo, el tratamiento del *HALT* y el drenado del *pipeline* evolucionaron hacia un esquema más robusto, basado en la detección efectiva del vaciado de los registros inter-etapa, lo que permitió garantizar una finalización consistente de la ejecución.

A lo largo del desarrollo surgieron además distintas dificultades prácticas que enriquecieron el alcance formativo del trabajo. Entre ellas se destacaron el ajuste de la latencia efectiva de las memorias para alinearla con lo esperado por el *pipeline*, la diferenciación conceptual entre pausa externa y terminación interna, y la validación del protocolo UART en escenarios de comunicación más extensos, como los volcados de registros, latches y memoria. La resolución de estos problemas llevó a un diseño más claro, modular y coherente, y obligó a adoptar criterios de validación más precisos tanto en simulación como en hardware real.

En conjunto, los resultados obtenidos indican que el sistema desarrollado cumple con los objetivos propuestos y constituye una implementación sólida y funcional del procesador requerido. Más allá del cumplimiento del trabajo práctico, el proyecto integró conocimientos de diseño digital, arquitectura de computadoras, verificación, temporización e interacción entre hardware y software, ofreciendo una experiencia de desarrollo completa sobre FPGA.